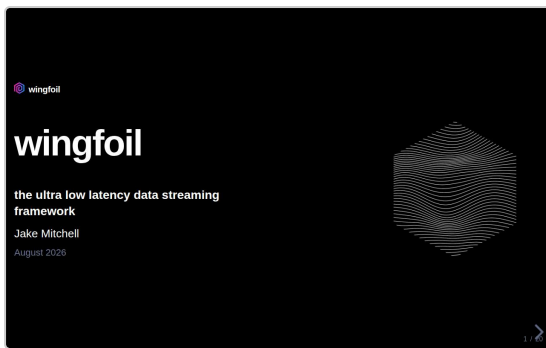


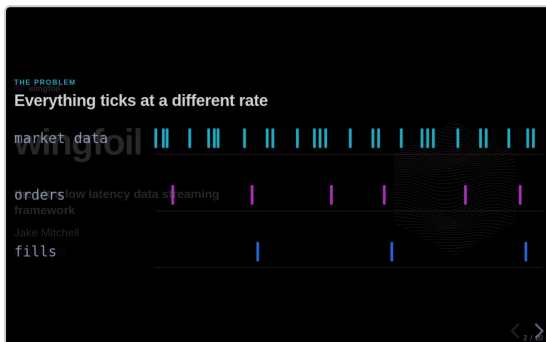
wingfoil — speaker notes

20 slides · printed fallback for the reveal.js speaker view (press S)



1 / 20

- Thanks for coming. Jake Mitchell. Open source project — **wingfoil**.
- Three parts:
- **1** — background: myself, the problem, and why you need wingfoil to solve it.
- **2** — **Nitro**. The feature we released last week. Why we wanted it. Why it meant rewriting the entire framework. The pattern we found, and what it delivered.
- **3** — what's next.
- —
- 25 years in financial services.
- Mostly as a quant at JPMorgan — building electronic trading systems for options: across interest rates, FX, precious metals.
- Now at B2C2, working for their crypto options desk.
- I learnt what the most effective patterns were for building these kinds of systems.
- I wanted to bring them into the open source world. And I wanted to use **Rust** to build it.

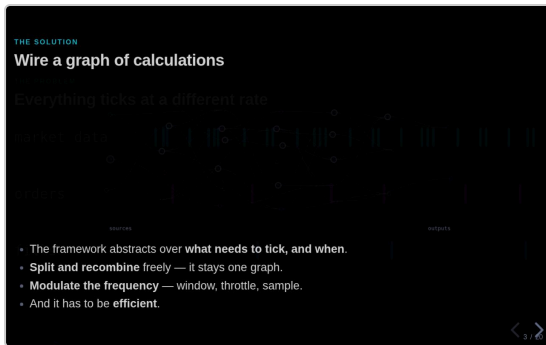


2 / 20

THE PROBLEM

Everything ticks at a different rate

- The problem.
- You're building something that reacts to information that is arriving at **different rates**.
- For example — market data ticking very frequently; orders and fills at a different frequency.
- This could be across tens of thousands of assets.
- And you need to react fast — at micro or even **nanosecond** time scales.

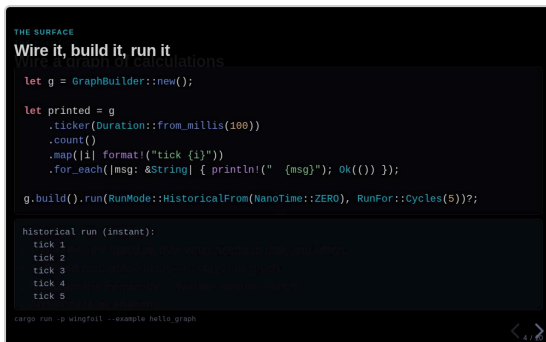


3 / 20

THE SOLUTION

Wire a graph of calculations

- Wire a graph of calculations.
- Have a framework that **abstracts over what needs to tick at what time**.
- That allows you to split and recombine.
- And to modulate their frequency — for example window or throttle.
- And is **efficient**.

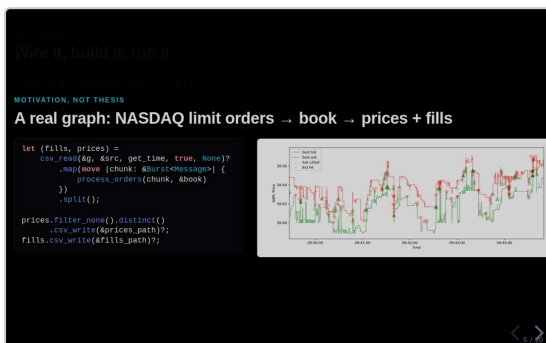


4 / 20

THE SURFACE

Wire it, build it, run it

- This is what it looks like in practice.
- This example is a simple linear pipeline, with every node ticking on each engine cycle.

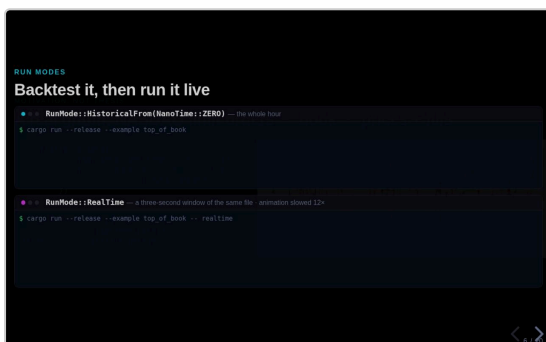


5 / 20

MOTIVATION, NOT THESIS

A real graph: NASDAQ limit orders → book → prices + fills

- Here is a more interesting example: a **matching engine**.
- Replay limit orders from file.
- Maintain an order book over time.
- Produce top-of-book prices and fills.
- **Three different frequencies of data** — many messages share a timestamp, top of book changes less often, trades are sparser still. Nothing is coalesced or dropped to make them agree.

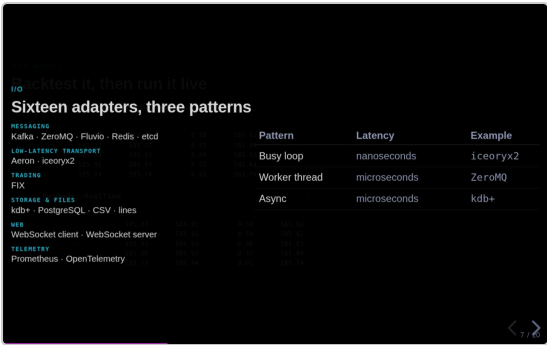


6 / 20

RUN MODES

Backtest it, then run it live

- Wingfoil supports **realtime** and **historical** modes.
- **Historical** [top] — development · unit tests · integration tests · back test.
- **Realtime** [bottom] — production.
- Same wiring. Same graph. **One line changes** — the run mode.

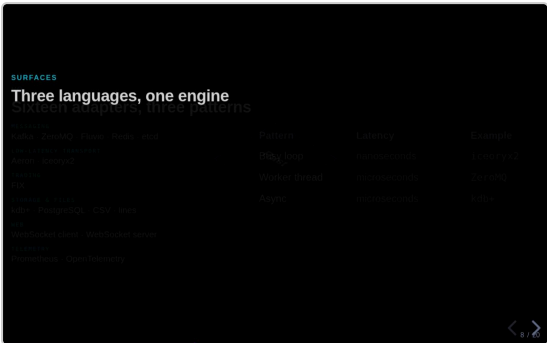


7 / 20

I/O

Sixteen adapters, three patterns

- We've built a bunch of I/O adapters — FIX, kdb+, iceoryx2, and more.
- They use these three I/O patterns:
 - **Busy spin** — for latency-critical, realtime use cases.
 - **Dedicated worker thread** with waker pattern.
 - **Async** — but only where you need it, at the graph edge. The graph itself is synchronous.
- You can **mix and match** — choose the pattern that works for your use case. The core stays synchronous either way, so **you are not locked into async**.

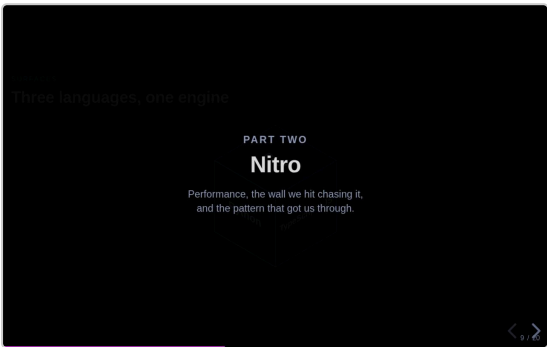


8 / 20

SURFACES

Three languages, one engine

- **Rust** — the best language to work with.
- Also **Python** bindings, for those that have yet to make the jump.
- **TypeScript** integrations, to stream into the browser.



9 / 20

PART TWO

Nitro

- That's part one.
- Part two — Nitro.

THE ENGINEERING PROBLEM

What we wanted Nitro to factor out

Per node, per cycle, interpreted	What we wanted instead
a <code>Box<dyn FnMut></code> call	a direct call the optimiser can inline
a trip through a value slot	a local variable
a dirty-list walk to decide what runs	straight-line code, in topological order

What we wanted Nitro to factor out

- The interpreted engine is fine, but there's overhead I don't love. Four things, per node, per cycle.
- A **boxed dyn call**. A **trip through a value slot**. A **dirty-list walk** to work out what runs.
- And the one that isn't on the slide: an **optimiser barrier at every node** — LLVM can't see across a dyn boundary, so nothing folds or inlines between nodes. Compiled, it's one function seen whole.
- On a hot chain that's all bigger than the arithmetic you came to do.
- Obvious answer: **generate the code**. Straight-line function, state in locals, optimiser across node boundaries. Every ingredient exists.
- So we built it. Wiring at build time, walk the graph, emit a runner. **It failed. Three ways.**
- **1** — it had to **re-implement every node**. Semantics were welded to `RefCell` fields, so generated code couldn't *call* a node's cycle. Two maps, two filters, nothing keeping them in step.
- **2** — **types and closures were already gone**. You cannot get `|x| x * 10` back out of a `Box<dyn Fn>`. The workaround was the user retyping every closure in a side table. *[Usually gets a laugh. Let it.]*
- **3** — **scheduling was inferred from names**. Nothing declared "I schedule myself", so the engine drove nodes off allowlists.
- We deleted it. *[Pause.]*
- **The problem was never the generator**. Node semantics weren't separately callable. No cleverness in an emitter fixes a design with nothing to emit a call *to*. That's why it meant rewriting the framework.

You write the wiring once, in plain Rust

- You write the wiring once. Plain Rust.
- Real function. Valid, reviewable, readable in a PR.
- Out the other side: interpreted, compiled, nested. **Same tokens.**
- `count` used three times → a shared node. This is a real DAG, not a chain.
- *[Constraint, if asked: wiring must be straight-line. Values and closure logic can be as procedural as you like — the TOPOLOGY can't depend on a runtime value, because compiled monomorphizes one local per node.]*

One function. One local per node. No dyn.

- What it turns into.
- Kernel on the stack. One local per node — `cfg`, `state`, `value`. Then a loop.
- *[Point at the call.] That's the same cycle function the interpreter calls.* Not a copy. A call.
- Wall number one, dead.
- Error context = the binding name *you* wrote, baked in as a static string. Costs nothing until something breaks.
- Whole expansion is committed. 1,730 lines. Don't take my word for it.

Semantics became an associated function

- Four associated types and a function.
- `Cfg` (closures live here) · `State` (engine owns) · `In` (typed, per cycle) · `Out`. Plus a `const` for scheduling.
- **Notice what's missing.** Nothing says where state lives. Nothing says who calls it.
- It's a free function over explicit arguments.
- *[Return type, if asked: Tick has THREE states. Value ticks downstream, Quiet does nothing, Silent updates the value without ticking — what delay needs.]*

THE FRONT DOOR

You write the wiring once, in plain Rust

```
wiringfoil::nitro! {
  fn odds_events(g: &graphBuilder) -> Stream<String> {
    let count = g.ticker(PERIOD).count();
    let is_even = count.map(|i| i.is_multiple_of(2));
    let is_odd = is_even.map(|b| !b);
    let odd_str = count.filter(&is_odd).map(|i| format!("{i} is odd"));
    let even_str = count.filter(&is_even).map(|i| format!("{i} is even"));
    let labelled = odd_str.merge(&even_str);
    labelled
  }
}
```

...AND THIS IS WHAT IT BECOMES

One function. One local per node. No dyn.

```
let mut _k = kernel::new(run_node, run_for);

let mut _cfg_wf_anon_1 = PERIOD;
let mut _state_wf_anon_1 = _wf_op.ticker_seed_state(&_cfg_wf_anon_1);
let mut _v_wf_anon_1 = _wf_op.ticker_seed_value(&_cfg_wf_anon_1);
// ... one Cfg / State / value local per node, 11 nodes ...

let mut _dirty = [false; 11usize];
while _k.main_cycle(&mut _dirty) {
  let _t_wf_anon_1 = // activation /> && {
    let mut _ctx = Ctx::new(&mut _k, &mut _state_wf_anon_1, &mut _v_wf_anon_1, &mut _dirty);
    match _wf_op.ticker_cycle(&mut _cfg_wf_anon_1, &mut _state_wf_anon_1, &mut _v_wf_anon_1, &mut _dirty) {
      .context("Node 0 (wf_anon_1.ticker) cycle?") {
        Tick::Value(_val) => { _v_wf_anon_1 = _val; true }
        Tick::Silent(_val) => { _v_wf_anon_1 = _val; false }
        Tick::Quiet => false,
      }
    };
    // ... 10 more, omitted in topological order ...
  };
}
```

shrugged - millions marked - All 1,730 lines committed verbatim below the example.
github.com/wingfoil-io/wingfoil - examples/core/dual_node/expanded/

THE ONE DECISION EVERYTHING FOLLOWS FROM

Semantics became an associated function

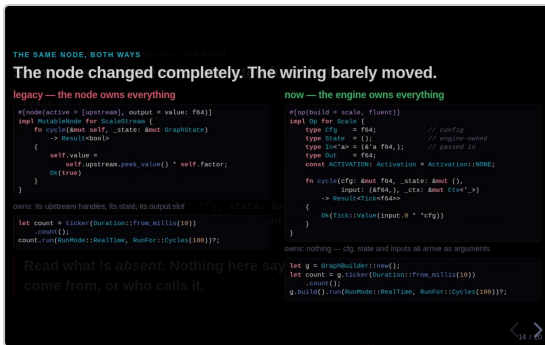
```
trait Op {
  type Cfg;           // construction-time config; closures live here
  type State;         // engine-owned mutable state
  type In<'a>;        // typed inputs, passed in per cycle
  type Out;           // the produced value
  const ACTIVATION: Activation;

  fn cycle(cfg: &mut Self::Cfg, state: &mut Self::State,
    input: Self::In<'a>, ctx: &mut Ctx<'a>) -> Result<Tick<Self::Out>>;
}
```

Read what is absent. Nothing here says how state is stored, where inputs come from, or who calls it.

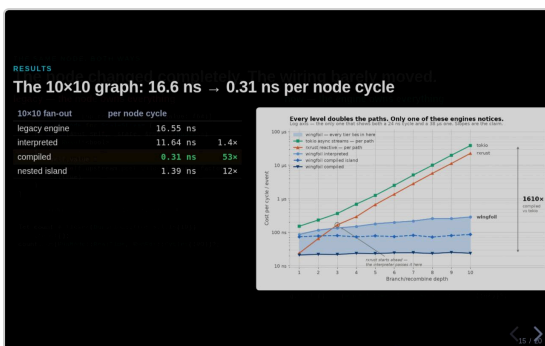
The node changed completely. The wiring barely moved.

- Same node, both ways.
- **Left:** `&mut self`, peeks a stored upstream handle, writes its own field. Owns its upstreams, its state, its output slot.
- **Right:** same node as an op. Owns **nothing** — everything arrives as an argument.
- Those two are unrecognisable.
- **Now the bottom row** — the wiring the user writes. Before and after.
- **One line different.** You hold a builder and make sources on it.
- We turned the engine inside out, and it cost anyone using it one line.



The 10×10 graph: 16.6 ns → 0.31 ns per node cycle

- The 10×10 fan-out — 103 nodes, every one ticking, ten thousand cycles. All four tiers measured back to back in one run.
- Old engine **16.6 nanoseconds** per node cycle. Compiled: **0.31**. That's **53×**.
- Against the new interpreted engine it's 37×. The island lands between, as it should.
- The one I care about most: **new interpreted beats the OLD engine** — on all eight workloads. Most graphs keep running interpreted, and a rewrite that trades everyday performance for a fast path nobody uses is a bad trade.
- Chart: branch and recombine, paths doubling every level. Rx and async streams pay **per path** — slopes 2.01 and 1.94. We pay per node, so we're flat.
- *[Caveats if pushed: rxrust figure is HALVED from their raw number — their bench emits twice per sample. Tokio target is recursive Box::pin over a ready leaf: honest for per-path propagation, not the best async could do. **The claim is the slope, not the multiplier.**]*
- *[Read the ratios, not the times. Shared cloud VM.]*



The 10x10 graph: 16.6 ns — 0.31 ns per node cycle

Component	Time (ns)
Verilog / FPGA	16.6
Core pin	1.0
Kernel bypass	1.0
Zero-allocation I/O	1.0

PART THREE

What's next

16 / 20

PART THREE

What's next

- That's Nitro.
- Last bit — where this goes.

Closer to the metal

- 1 Verilog / FPGA — the graph as gateway, with the backtest as its testbench.
- 2 Core pin — the graph thread on an isolated core.
- 3 Kernel bypass — Onload, then ef_vi/DPDK. This is the one that buys the wire-to-trade number we currently decline to claim.
- 4 Zero-allocation I/O — no allocator on the hot path.

That's next

17 / 20

Closer to the metal

- **Verilog** — the graph as gateway, backtest as testbench. Exploratory. Don't oversell it.
- **Core pin** — graph thread on an isolated core. Working implementation sits in an example; needs promoting into the runtime.
- **Kernel bypass** — Onload, then raw ef_vi/DPDK. Gets us a wire-to-trade number, which **I won't claim yet because I haven't measured it**.
- **Zero allocation** on the I/O path.
- The point: **none of these change your wiring**. Same graph definition, further down. That's the payoff from part two.

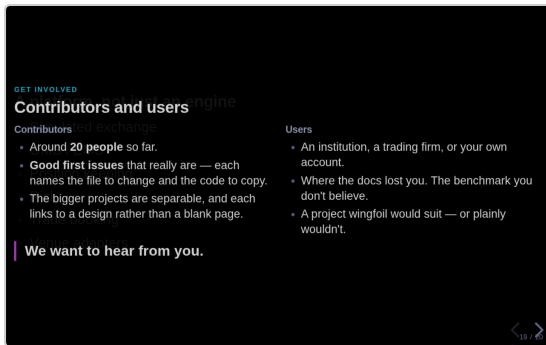
A platform, not just an engine

- Simulated exchange
- OMS / EMS
- Position keeping
- Risk
- Trade booking
- Venue adapters

18 / 20

A platform, not just an engine

- Where most of the value is, for most people.
- **Be straight:** wingfoil is a compute engine. It is *not* a trading platform.
- Missing: simulated exchange, OMS, position keeping, risk, booking, more venue adapters.
- Simulator is the hard one — a naive fill-at-touch simulator lies to you, and you'll believe it.
- **The bet:** none of it is a new kind of thing. Position keeping is a *fold*. Risk is a *filter*. PnL is a *join*. The sim venue is just an *op* — RunMode picks.
- Ordinary graph vocabulary. Not a second framework bolted on top.

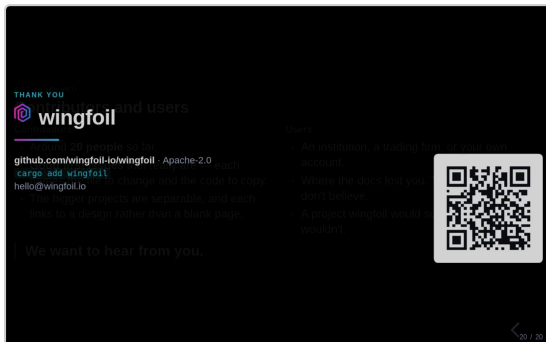


19 / 20

GET INVOLVED

Contributors and users

- The ask — and it's two asks.
- **Contributors.** ~20 so far. I'd love more.
- **The good first issues are actually good first issues** — each names the file to change and the code to copy. I know that label's been ruined elsewhere.
- Bigger projects are separable — one person each, and each links to a design, not a blank page.
- **Users.** Whether that's an **institution**, a **trading firm**, or you **trading your own account** — we want to hear from you.
- And it doesn't have to be code: where the docs lost you · which benchmark you don't believe · a project this would suit, or obviously wouldn't.
- **Please do get in touch.**



20 / 20

THANK YOU

wingfoil

- Thank you.
- Repo's on the QR.
- *[Leave it up. Don't black the screen — let people scan.]*
- *[If it's quiet: the question I usually get is what happens when the graph shape isn't known until runtime. That's Project Lightning, and it's open.]*